

An End-to-End Lightweight Detection and Tracking Pipeline for UAV Imagery

Yukun Shi

Abstract—This paper presents an end-to-end aerial vision pipeline covering dataset analysis, object detection, single- and multi-object tracking, model compression, small-object ablation, knowledge distillation, and GPU deployment. Experiments use the VisDrone detection split and all 123 UAV123 tracking sequences. A full YOLOv8s detector reached 0.2933 mAP50-95, while a width-0.4375 student reduced checkpoint size by 22.4% and retained 0.2696 mAP50-95. SAHI raised slim-model small-object recall from 0.4868 to 0.6456 but reduced throughput from 96.2 to 10.0 FPS. Output-distribution distillation produced a small heavy-occlusion recall gain in the auxiliary group analysis but did not improve small-object recall. On UAV123, DaSiamRPN achieved 0.5564 success AUC at 39.9 FPS. The final TensorRT FP16 detector provided the best deployment trade-off: 0.2700 mAP50-95, 91.7 end-to-end FPS, and 1175.9 MiB peak process memory. INT8 was slower and less accurate on the evaluated GPU. These results show that aerial accuracy is governed primarily by target scale and occlusion, while effective deployment requires joint optimization of architecture, inference strategy, numerical precision, and tracking overhead.

Index Terms—unmanned aerial vehicles, object detection, object tracking, small objects, model compression, knowledge distillation, TensorRT.

I. INTRODUCTION

UAV imagery combines wide fields of view with small targets, dense traffic, camera motion, and frequent occlusion. The VisDrone benchmark formalized these difficulties for detection and tracking [1], while UAV123 provides 123 low-altitude single-object tracking sequences [2]. A practical airborne system must therefore balance localization accuracy, temporal stability, model size, latency, and memory rather than optimize a single leaderboard metric.

The project developed a reproducible pipeline from raw annotation analysis to a deployable detection-and-tracking application. YOLOv8 [3] was selected as the principal one-stage detector. Faster R-CNN [4] with an FPN [5] and a MobileNetV3 backbone [6] was also evaluated as an alternative lightweight direction. The study then tested resizing and color augmentation, conservative width reduction, focal loss [7], sliced inference [8], single-object Siamese tracking [9], online data association [10], [11], knowledge distillation [12], ONNX interchange [13], and TensorRT deployment [14].

The contributions are fourfold. First, the paper reports a complete dataset-to-deployment implementation using saved experimental artifacts rather than estimated values. Second, it isolates small-object and occlusion behavior with group-specific recall. Third, it compares classical and Siamese single-object trackers and integrates ByteTrack into the detector. Fourth, it documents negative results: aggressive slimming, untuned focal loss, image-level resampling, and the tested INT8 route did not improve the final accuracy-efficiency operating point.

II. RELATED WORK

A. Aerial Detection and Small Objects

VisDrone includes urban and suburban scenes with object categories, bounding boxes, truncation, and occlusion annotations [1]. Multi-scale representation is commonly addressed by FPN [5]. Focal loss reduces the contribution of easy examples under class imbalance [7], while SAHI enlarges the effective pixel footprint of small targets by applying a detector to overlapping slices [8]. COCO-style AP over multiple IoU thresholds is used as a localization-sensitive summary metric [15].

B. Tracking and Deployment

DaSiamRPN improves Siamese tracking robustness by explicitly learning distractor-aware features [9]. KCF and CSRT provide complementary classical correlation-filter baselines for speed-robustness comparison [16], [17]. For multi-object tracking, SORT demonstrated that detector quality strongly affects association quality [10], and ByteTrack later recovered low-score detections through a two-stage association strategy [11]. For deployment, ONNX provides a portable graph representation [13], whereas TensorRT performs graph optimization, layer fusion, kernel selection, and reduced-precision execution on NVIDIA GPUs [14].

III. DATA AND EXPERIMENTAL PROTOCOL

A. Datasets

The VisDrone training split contains 6,471 images and 353,550 annotated objects; the validation split contains 548 images and 40,169 objects. Objects with bounding-box area below 32 x 32 pixels account for 60.1% of training annotations and 67.7% of validation annotations. UAV123 contains 123 sequences and 112,578 annotated frames.

TABLE I
DATASET SUMMARY

Dataset	Images / Sequences	Objects / Frames
VisDrone train	6,471 images	353,550 objects
VisDrone validation	548 images	40,169 objects
UAV123	123 sequences	112,578 frames

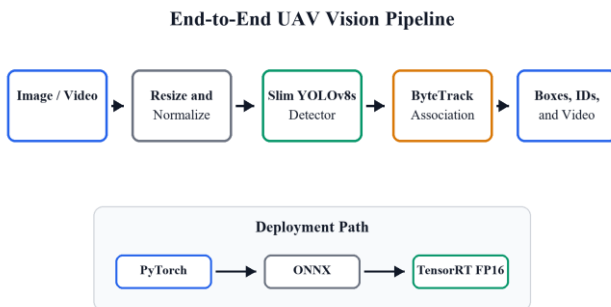


Fig. 1. Implemented processing and deployment pipeline. The teacher model is used only during training.

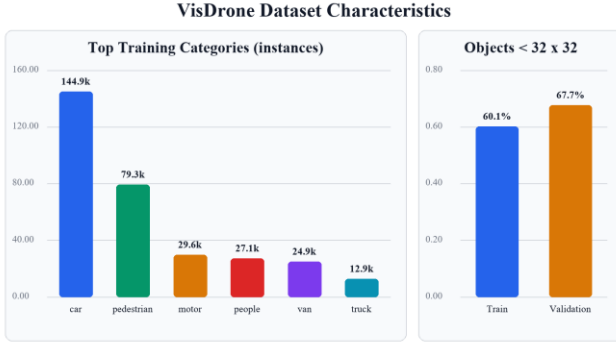


Fig. 2. Measured class imbalance and small-object prevalence in VisDrone.

B. Metrics and Hardware

Detection reports precision, recall, AP50, mAP50-95, and class-aware recall at IoU 0.50 for small objects and heavy occlusion. Single-object tracking reports success AUC, precision at 20 pixels, mean IoU, center error, and FPS. Deployment benchmarks use batch one after warm-up. All principal experiments ran in Python 3.10 with PyTorch and CUDA on an NVIDIA GeForce RTX 5080 Laptop GPU. TensorRT experiments used version 11.1. Results from distinct evaluator protocols are kept in separate comparisons.

IV. DETECTION, COMPRESSION, AND TRACKING RESULTS

A. Detector Development

The initial 10-epoch YOLOv8s baseline achieved 0.2551 mAP50-95. Extending the mild-augmentation recipe to the selected long-run checkpoint increased mAP50-95 to 0.2933 and small-object recall to 0.5201. A YOLOv8n model trained for 50 epochs was compact but reached only 0.2053 mAP50-95. An initial width-0.375 slim YOLOv8s also underperformed at 0.1687 mAP50-95. Conservative width-0.4375 scaling plus 100-epoch recovery produced an 8.62 M-parameter, 16.70 MiB checkpoint with 0.2696 mAP50-95.

TABLE II
UNIFIED VISDRONE DETECTOR RESULTS (AP = MAP50-95)

Model	P	R	AP50	AP	Small R
YOLOv8s e10	0.541	0.426	0.426	0.255	0.428
YOLOv8n e50	0.496	0.390	0.378	0.205	0.384
YOLOv8s full	0.598	0.487	0.492	0.293	0.520
Slim 0.4375	0.571	0.458	0.458	0.270	0.496
Distilled slim	0.572	0.464	0.459	0.270	0.495

The MobileNet direction did not replace YOLO. The aerial MobileNetV3-FPN baseline obtained 0.1033 mAP50-95, 0.2205 small-object recall, and 78.1 FPS. Class-balanced focal loss, small anchors, and small-box weighting raised small-object recall to 0.2426 and heavy-occlusion recall from 0.1909 to 0.2239, but reduced mAP50-95 to 0.0996 and throughput to 73.4 FPS. The result is a recall-oriented ablation, not a superior deployment model.

B. Single- and Multi-Object Tracking

All image-based single-object trackers were evaluated on 123 UAV123 sequences. DaSiamRPN achieved the highest success AUC and precision, while KCF was the fastest but least accurate. CSRT occupied the middle operating point.

The results support a learned Siamese tracker when robustness is more important than maximum throughput.

TABLE III
UAV123 SINGLE-OBJECT TRACKING

Tracker	AUC	P@20	Mean IoU	FPS
DaSiamRPN	0.556	0.718	0.555	39.9
CSRT	0.417	0.624	0.416	54.0
KCF	0.207	0.279	0.204	191.4

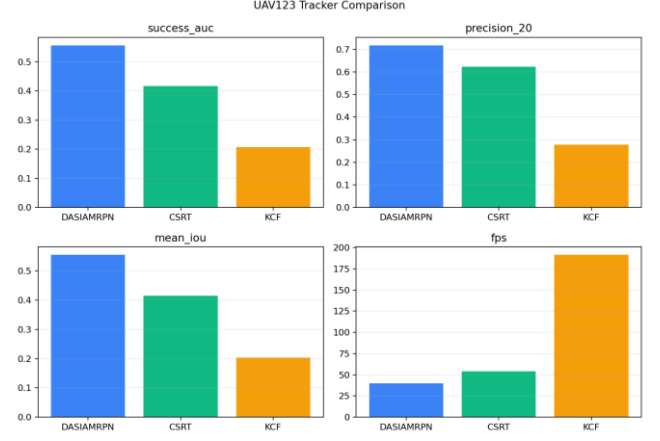


Fig. 3. Full UAV123 comparison of the three image-based single-object trackers.

The final detector-to-ByteTrack pipeline processed 300 frames from UAV123 group1, produced 1,070 track rows and 10 unique IDs, and ran at 43.05 FPS with TensorRT FP16. Because this sequence lacks complete multi-object identity ground truth, track counts and throughput are reported without inventing MOTA, IDF1, or HOTA values.

(a) Distilled detector validation predictions



(b) TensorRT FP16 and ByteTrack output



Fig. 4. Real validation predictions and integrated tracking output produced by the implemented pipeline.

V. SMALL-OBJECT AND DISTILLATION ABLATIONS

A. Small-Object Interventions

On the slim detector, multi-scale inference increased small-object recall from 0.4868 to 0.5907 at 20.98 FPS. SAHI reached 0.6456 small-object recall and 0.4888 mAP50, but throughput fell to 10.04 FPS. Focal loss raised recall to 0.5615 while slightly reducing mAP50-95. Image-level resampling produced 0.4861 recall, essentially no benefit. Thus SAHI is an offline accuracy mode, whereas standard inference remains the real-time mode.

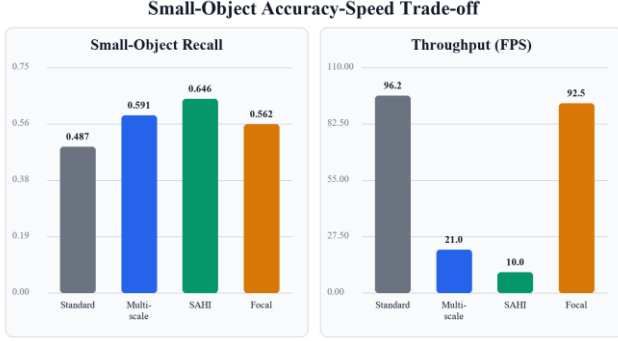


Fig. 5. Measured small-object recall and throughput for selected slim-detector ablations.

B. Teacher-Student Distillation

A full YOLOv8s teacher supervised aligned class logits and 16-bin bounding-box distributions from the slim student. The 20-epoch run preserved the student architecture. Recall increased from 0.4585 to 0.4640 and heavy-occlusion recall from 0.3611 to 0.3702 in the saved auxiliary group-recall summaries. However, mAP50-95 changed only from 0.2696 to 0.2699, and small-object recall changed from 0.4955 to 0.4953. Output-only distillation therefore helped occlusion modestly but did not transfer enough high-resolution feature information to improve tiny targets.

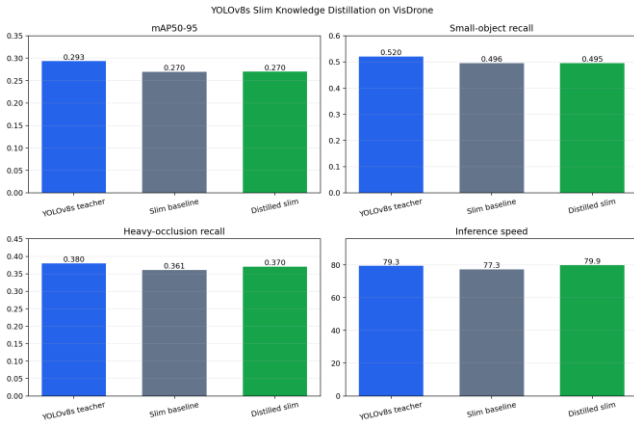


Fig. 6. Teacher, slim baseline, and distilled slim comparison from saved evaluation summaries.

VI. DEPLOYMENT RESULTS

PyTorch checkpoints were exported to ONNX and checked against framework predictions before TensorRT engine construction. In the full 548-image engine validation, FP16 achieved 0.2700 mAP50-95 and 477.2 raw engine FPS. Bright-scene INT8 calibration was the best of three calibration sets but reached only 0.2467 mAP50-95 and 333.9 FPS. Restoring the detection head to FP16 recovered

mAP50-95 to 0.2617, confirming that classification, DFL, and decode layers were quantization-sensitive, but the hybrid still underperformed full FP16.

TABLE IV
TENSORRT ACCURACY AND ENGINE THROUGHPUT (AP = MAP50-95)

Engine	P	R	AP50	AP	Raw FPS
FP16	0.561	0.466	0.458	0.270	477.2
INT8 bright	0.541	0.442	0.433	0.247	333.9
Head FP16	0.557	0.450	0.445	0.262	361.6

An isolated end-to-end runtime benchmark, which includes the Ultralytics prediction path and post-processing, measured 91.7 FPS and 1175.9 MiB peak process RAM for TensorRT FP16. INT8 was both slower at 78.6 FPS and larger in memory at 1192.5 MiB. Adding ByteTrack reduced throughput to 63.8 FPS and increased peak RAM to 1215.0 MiB. The difference from raw engine FPS is expected because the end-to-end protocol includes preprocessing, result construction, NMS, and tracking.

Deployment Accuracy and End-to-End Runtime

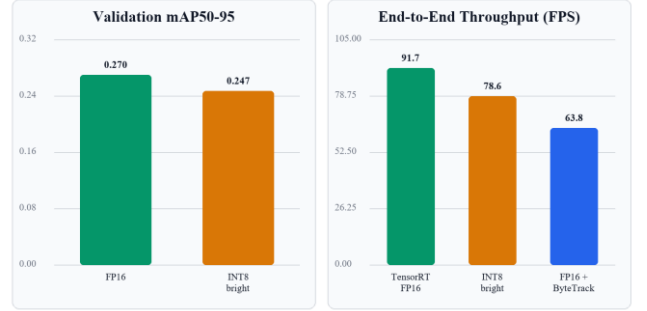


Fig. 7. TensorRT validation accuracy and end-to-end throughput. The two panels use their stated protocols.

VII. DISCUSSION AND LIMITATIONS

The experiments identify a stable hierarchy of constraints. First, spatial resolution dominates small-object recall: multi-scale and sliced inference gave the largest gains, but every gain required multiple forward passes. Second, moderate width reduction preserved much more accuracy than aggressive slimming, showing that recovery training cannot compensate for an undersized feature hierarchy. Third, tracking quality remained detector-dependent; ByteTrack made the pipeline usable, but its temporal accuracy cannot be claimed without identity-complete UAV ground truth. Fourth, reduced precision was hardware- and graph-dependent. INT8 did not automatically improve latency and was especially sensitive in the detection head.

Several limitations bound the conclusions. All compute measurements were made on one laptop GPU rather than a Jetson-class target, and WDDM prevented reliable per-process VRAM attribution. VisDrone validation metrics are produced by project evaluators and are not official challenge-server test scores. Some ablation tables use different score-retention rules and are compared only internally. The current distillation loss operates on output distributions rather than ground-truth-masked P3 features. Finally, the multi-object demonstration measures engineering completeness and throughput, not MOTA or IDF1.

VIII. CONCLUSION

The completed system demonstrates a reproducible UAV perception workflow from data understanding to deployment. Full YOLOv8s provided the best detector accuracy, while the width-0.4375 student offered a useful storage-accuracy compromise. DaSiamRPN was the strongest evaluated single-object tracker, and ByteTrack enabled an integrated multi-object pipeline. SAHI substantially improved small-target recall but was not real-time. Knowledge distillation partially recovered occlusion recall without changing the student architecture. TensorRT FP16 was the final deployment choice because it outperformed the tested INT8 engines in accuracy, speed, and memory. Future work should evaluate P3 feature distillation, adaptive slicing, identity-labeled aerial MOT sequences, and sustained thermal performance on the actual embedded device.

REFERENCES

- [1] P. Zhu, L. Wen, X. Bian, H. Ling, and Q. Hu, "Vision Meets Drones: A Challenge," arXiv:1804.07437, 2018.
- [2] M. Mueller, N. Smith, and B. Ghanem, "A Benchmark and Simulator for UAV Tracking," in Proc. ECCV, 2016, pp. 445-461, doi: 10.1007/978-3-319-46448-0_27.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," version 8.0.0, software, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in Adv. Neural Inf. Process. Syst., vol. 28, 2015.
- [5] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in Proc. IEEE CVPR, 2017, pp. 2117-2125.
- [6] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF ICCV, 2019, pp. 1314-1324.
- [7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," in Proc. IEEE ICCV, 2017, pp. 2980-2988.
- [8] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing Aided Hyper Inference and Fine-Tuning for Small Object Detection," in Proc. IEEE ICIP, 2022, pp. 966-970, doi: 10.1109/ICIP46576.2022.9897990.
- [9] Z. Zhu, Q. Wang, B. Li, W. Wu, J. Yan, and W. Hu, "Distractor-Aware Siamese Networks for Visual Object Tracking," in Proc. ECCV, 2018, pp. 101-117.
- [10] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in Proc. IEEE ICIP, 2016, pp. 3464-3468, doi: 10.1109/ICIP.2016.7533003.
- [11] Y. Zhang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Proc. ECCV, 2022, pp. 1-21, doi: 10.1007/978-3-031-20047-2_1.
- [12] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," arXiv:1503.02531, 2015.
- [13] ONNX Working Group, "Open Neural Network Exchange Intermediate Representation Specification," version 1.23.0, 2026. [Online]. Available: <https://onnx.ai/onnx/repo-docs/IR.html>. Accessed: Jul. 28, 2026.
- [14] NVIDIA, "NVIDIA TensorRT Documentation," version 11.1.0, 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/>. Accessed: Jul. 28, 2026.
- [15] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Proc. ECCV, 2014, pp. 740-755.
- [16] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-Speed Tracking with Kernelized Correlation Filters," IEEE Trans. Pattern Anal. Mach. Intell., vol. 37, no. 3, pp. 583-596, 2015, doi: 10.1109/TPAMI.2014.2345390.
- [17] A. Lukezic, T. Vojir, L. Cehovin Zajc, J. Matas, and M. Kristan, "Discriminative Correlation Filter with Channel and Spatial Reliability," in Proc. IEEE CVPR, 2017, pp. 6309-6318.